Machine Learning

Concepts, workflows, evaluation, and practical case studies

Purpose of this document

This document is designed as a realistic source file for testing document ingestion and retrieval systems. It combines explanatory prose, practical examples, comparison tables, metrics, and related concepts that deliberately overlap across sections. A good retrieval pipeline should locate the passage that best answers a question, preserve table rows, and return a useful citation to the source document.

How to use it

- Ingest the PDF into a vector database and confirm that extracted text and tables survive conversion.
- Try exact queries such as “stratified split”, “F1 score”, “learning rate”, and “data leakage”.
- Try conceptual queries such as “why can a high accuracy model still be poor?” and compare semantic, keyword, and hybrid retrieval.
- Inspect whether tables are preserved as coherent Markdown or equivalent structured text, rather than split across unrelated chunks.
- Use the source filename and page-level metadata, when available, to verify citations.

Contents at a glance

Section	Coverage
1. Foundations and problem framing	What machine learning is, learning settings, data splits, and leakage.
2. Data preparation and features	Missing values, categorical encoding, scaling, imbalance, and feature design.
3. Supervised learning algorithms	Linear models, trees, ensembles, SVMs, and nearest neighbours.
4. Neural networks and training	Losses, gradients, regularization, and learning-rate choices.
5. Evaluation and model selection	Classification and regression metrics, validation, and threshold decisions.
6. Unsupervised learning	Clustering, dimensionality reduction, and anomaly detection.
7. Deployment and monitoring	Pipelines, reproducibility, drift, and practical case studies.

Note: examples and values in this document are illustrative teaching examples, not claims about a particular production system.

1. Foundations and problem framing

Machine learning (ML) is a way to build systems that infer patterns from examples and use those patterns to make predictions or decisions. Instead of writing a separate rule for every possible input, a developer selects a learning objective, supplies representative data, and trains an algorithm to minimize a defined error. The trained model is then evaluated on data that was not used to fit its parameters. The goal is not to memorize the training examples; the goal is to generalize to new examples drawn from the same intended problem setting.

A project begins by translating a broad business question into a measurable prediction task. “Reduce customer churn” is not yet a model specification. A more concrete formulation might predict whether a customer will cancel within the next 30 days using information available today. This statement defines a target, prediction horizon, and a time boundary for the features. Those details matter: if the model accidentally uses a cancellation date recorded after the prediction moment, its validation score may look excellent while its real-world performance is unusable.

Learning settings

In supervised learning, each example includes an input and a target label or numeric value. Classification predicts a category, such as whether a transaction is fraudulent. Regression predicts a quantity, such as a house price. Unsupervised learning works without target labels and instead looks for structure, for example grouping customers by behaviour. Reinforcement learning is different again: an agent takes actions, observes outcomes, and learns a policy that maximizes accumulated reward. These settings are related but not interchangeable, because they require different datasets, objectives, and evaluation protocols.

Setting	Training signal	Example task	Common evaluation
Classification	Categorical label	Predict whether an email is spam	Precision, recall, F1, ROC-AUC
Regression	Numeric target	Estimate delivery time in minutes	MAE, RMSE, R-squared
Clustering	No provided target label	Group customers by purchase patterns	Silhouette score plus domain review
Dimensionality reduction	Structure in feature values	Compress many correlated variables	Reconstruction error or downstream performance
Reinforcement learning	Rewards from sequential actions	Learn a policy for inventory decisions	Cumulative reward and safety constraints

Problem framing checklist

- Unit of prediction: define whether one row represents a customer, device, event, image, or time interval.
- Target and horizon: specify exactly what is predicted and when the prediction is made.
- Available information: use only features that will be known at prediction time.
- Cost of errors: decide whether false positives, false negatives, or numeric errors are most costly.
- Baseline: measure a simple rule or naive model before investing in a complex algorithm.

2. Data preparation, splitting, and leakage

Model quality depends heavily on the data representation. Before selecting an algorithm, teams inspect the number of rows and columns, data types, duplicates, missingness, target distribution, and suspicious relationships among features. Data preparation is not simply a cleaning step at the beginning: each transformation must be fitted using training data and then applied consistently to validation, test, and future production data. A preprocessing pipeline prevents small differences between training and inference from becoming hidden sources of error.

Train, validation, and test sets have distinct jobs. The training set estimates model parameters. The validation set supports choices such as feature selection, hyperparameters, thresholds, and model comparisons. The final test set estimates performance after those decisions are complete. Repeatedly choosing models based on test results leaks information from the test set into development and makes the final estimate optimistic. For time-dependent data, a random split may be inappropriate; training on the future and evaluating on the past can produce an unrealistic result.

A subtle but serious issue is data leakage. Leakage occurs when a feature, preprocessing step, or split strategy introduces information that would not legitimately be available at prediction time. Leakage can be direct, such as including a target-derived column, or indirect, such as standardizing the entire dataset before splitting. Even though standardization does not use the target, it uses information about the future validation or test distribution. The safer procedure is to fit the scaler on training rows only and reuse the learned mean and standard deviation for other splits.

Split strategy	Use it when	Main caution
Random train/test split	Rows are independent and identically distributed enough for random sampling	Near-duplicates or groups may appear across splits
Stratified split	Classification labels are imbalanced and each partition should preserve class proportions	It does not solve time leakage or group leakage
Group split	Several rows belong to the same person, account, patient, or device	All rows from a group should remain in one partition
Temporal split	Future predictions are the actual use case	Respect the time order and use only past information
K-fold cross-validation	Data is limited and several train/validation rotations are useful	Use stratified, group-aware, or time-aware variants when needed

Example: a hospital model predicts readmission. If rows from the same patient occur in both training and test sets, the model may recognize patient-specific patterns instead of learning transferable risk signals. A group split by patient gives a more realistic estimate. If the target concerns future readmission, all input features must also be restricted to information recorded before the prediction timestamp.

Data preparation should be reproducible. Save the split definition, random seed, feature list, preprocessing parameters, model configuration, and dataset version. Without those details, a score cannot be reliably reproduced or compared with a later experiment.

3. Missing values, categorical variables, and scaling

Real datasets commonly contain missing values, inconsistent categories, mixed units, and noisy measurements. A missing value may indicate a failed sensor, a question that was not asked, or a meaningful absence. Replacing every missing value with zero is dangerous because zero may be a valid measurement. The correct method depends on the feature meaning, the amount and pattern of missingness, and what will be available at prediction time.

Simple imputation uses statistics such as the mean, median, or most frequent category. Median imputation is often more robust than mean imputation when numeric data contains outliers. A missing-indicator feature can preserve the fact that a value was absent, which can be predictive when missingness has a systematic cause. More elaborate approaches, including K-nearest-neighbour imputation and iterative imputation, estimate missing values using relationships among features; these methods may help but are more computationally expensive and can add assumptions.

Feature issue	Common approach	Example / caution
Numeric missingness	Median imputation	A salary column is skewed by a few very high incomes.
Categorical missingness	Most-frequent or explicit “Unknown” category	“Unknown” can be meaningful; do not silently merge it with a common value.
Different numeric scales	StandardScaler or MinMaxScaler	Distance-based algorithms can be dominated by a feature measured in thousands.
Nominal categories	One-hot encoding	Red, blue, and green have no natural numeric order.
Ordered categories	Ordinal encoding when order is justified	Low, medium, high have a meaningful order, but spacing is not necessarily equal.
Rare categories	Category grouping or carefully designed encoding	Grouping should be fitted using training data and preserve useful distinctions.

Feature scaling changes numeric values without necessarily changing their underlying meaning. Standardization subtracts the training mean and divides by the training standard deviation. Min-max scaling maps values to a chosen interval, often zero to one. Scaling is particularly important for algorithms based on distances or gradient optimization, including K-nearest neighbours, support vector machines, and many neural networks. Decision trees and random forests usually do not require scaling because their split decisions depend on feature thresholds rather than comparable geometric distances.

A robust preprocessing pipeline keeps transformations tied to the model. For example, a scikit-learn ColumnTransformer can apply median imputation and standardization to numeric columns while applying most-frequent imputation and one-hot encoding to categorical columns. Putting the transformer inside a Pipeline means cross-validation fits it independently on each training fold, reducing leakage risk.

Practical example

A house-price dataset includes floor area, number of bedrooms, neighbourhood, and property type. Floor area is numeric and may need scaling for a distance-based regressor. Neighbourhood and property type are categorical and should not be encoded as arbitrary integers, because that could incorrectly imply that neighbourhood 8 is greater than neighbourhood 3 in a meaningful numeric sense. Missing bedroom counts might be imputed, but the team should first investigate whether missingness reflects incomplete records or a separate property category.

4. Supervised learning algorithms

No single supervised algorithm is best for every dataset. Linear models provide a useful baseline and are often easier to interpret. Decision trees learn a sequence of feature-based splits. Ensembles combine multiple models to reduce variance or improve predictive strength. Support vector machines can create decision boundaries with a margin, while K-nearest neighbours predicts from the labels or values of similar training examples. Algorithm choice should be tied to sample size, feature types, noise, latency, interpretability, and evaluation results.

Linear regression estimates a numeric target as a weighted combination of features. Its simplicity is helpful, but its standard form may not capture complex nonlinear relationships without engineered features or transformations. Logistic regression is commonly used for classification even though its name includes regression: it estimates class probabilities through a logistic function, and a decision threshold converts the probability into a predicted class. The threshold need not be 0.5; a business may choose a different value to balance false positives and false negatives.

A decision tree recursively partitions the feature space into regions that are relatively homogeneous with respect to the target. Trees can model interactions and nonlinear boundaries and are easy to visualize when shallow. Deep trees, however, can memorize training examples, leading to high variance. Maximum depth, minimum samples per leaf, and pruning can reduce overfitting. Random forests train many trees using bootstrapped samples and random subsets of features; averaging their predictions usually improves stability compared with a single unconstrained tree.

Algorithm	Strengths	Limitations	Typical preparation
Linear / logistic regression	Fast baseline; coefficients can be inspected	May miss nonlinear patterns; sensitive to correlated features in interpretation	Scale numeric data for some use cases; encode categories
Decision tree	Nonlinear rules; limited scaling needs; interpretable when small	Can overfit; unstable to small data changes	Handle categories according to library capabilities
Random forest	Strong general baseline; captures interactions; robust across many tabular tasks	Larger models; explanations less direct; may be weak at extrapolation	Usually no scaling required
Gradient boosting	Often strong on structured/tabular data	Needs careful tuning; may overfit noisy small datasets	Manage missing values and categories according to implementation
Support vector machine	Effective in high-dimensional spaces; kernels model nonlinear boundaries	Can be expensive at large sample sizes; parameter-sensitive	Scaling is commonly important
K-nearest neighbours	Simple; flexible local decision boundaries	Prediction can be slow; sensitive to distance and irrelevant features	Scale features; choose distance and neighbour count

A useful experiment starts with a simple baseline, then compares a small number of justified alternatives under the same split and metric. Changing the algorithm, split, preprocessing, and threshold simultaneously makes it difficult to identify which decision caused a performance change.

5. Decision trees, ensembles, and error patterns

Ensemble learning combines predictions from several models. Bagging trains models on different resampled datasets and averages or votes across them. Random forests add feature randomness to decorrelate the trees. Boosting builds models sequentially, with later models emphasizing errors that earlier models handled poorly. These mechanisms are not identical: bagging mainly reduces variance, while boosting can reduce bias and build a highly flexible predictor, although it may overfit if tuning is poor.

Consider a dataset that predicts equipment failure from temperature, vibration, load, and operating age. A single tree might create a rule that triggers when vibration exceeds a threshold and temperature is high. A random forest may combine many different thresholds and feature subsets, making the output less sensitive to one unusual training sample. A gradient-boosted model may capture smaller residual patterns left by previous trees. In all three cases, evaluation must use a split that resembles the equipment the model will face after deployment.

Feature importance can help investigate a model, but it should not be mistaken for a causal explanation. A feature may appear important because it acts as a proxy for another variable, because it correlates with the target due to leakage, or because correlated features share importance. Permutation importance estimates how performance changes when a feature is disrupted, while SHAP-style methods attribute a prediction relative to a defined reference distribution. Both require careful interpretation and should be checked against domain knowledge.

Observed symptom	Possible cause	Useful check
Very high training score, weak validation score	Overfitting, leakage, or distribution mismatch	Compare train/validation errors, audit features and split strategy
Large performance variation across folds	Small dataset, rare classes, or unstable split	Inspect fold composition and confidence intervals
Good average score, poor minority-class recall	Class imbalance or unsuitable threshold	Inspect per-class metrics and confusion matrix
Model fails on new sites or devices	Domain shift or group leakage in validation	Use group/site split and evaluate subgroup performance
Tree ensemble has suspiciously perfect accuracy	Target proxy or post-outcome feature	Trace when each feature becomes available

Hyperparameters are settings chosen before or around training, such as tree depth, number of trees, regularization strength, learning rate, or number of neighbours. Parameters are values learned from the data, such as linear-model coefficients. Grid search evaluates specified combinations; random search samples combinations; Bayesian optimization uses prior experiment outcomes to propose promising configurations. More search does not automatically produce a better model if the evaluation data is small or repeatedly reused.

A sound model-selection report records the baseline, candidate models, hyperparameter ranges, selected metric, validation method, and final test score. This record helps other engineers reproduce the decision and distinguish genuine improvements from changes caused by random splitting.

6. Neural networks and optimization

A neural network transforms input features through layers of learned weights and nonlinear activation functions. During a forward pass, the network produces a prediction. A loss function compares that prediction with the target and returns a numeric measure of error. Backpropagation computes gradients of the loss with respect to the parameters, and an optimizer updates those parameters to reduce the loss over repeated batches of examples. A model can contain many layers, but the training loop still revolves around predictions, loss, gradients, and parameter updates.

For regression, mean squared error penalizes the square of each prediction error and therefore gives large errors a strong influence. Mean absolute error penalizes absolute differences and is often easier to interpret in the original target units. For classification, cross-entropy is a standard loss when the model predicts class probabilities. The loss function used for training is not necessarily the only metric used for reporting; a fraud system might train with cross-entropy but be selected based on recall at an acceptable false-positive rate.

Gradient descent updates parameters in the direction that reduces loss. Full-batch gradient descent uses all training data for a step; stochastic gradient descent uses one example; mini-batch gradient descent uses a small subset and is common in deep learning. The learning rate controls update size. Too large a learning rate can make training unstable or cause the loss to diverge. Too small a rate can make learning slow or stall before a useful solution is reached. Schedulers may reduce the learning rate as training progresses.

Training concept	What it controls	Warning sign	Possible response
Learning rate	Size of parameter updates	Loss oscillates or becomes NaN	Lower learning rate; check input scale and gradients
Batch size	Examples processed per update	Noisy updates or excessive memory use	Adjust batch size based on compute and generalization
Epoch	One full pass through training data	Training continues improving while validation worsens	Use early stopping or stronger regularization
Dropout	Randomly disables activations during training	Training/validation gap is large	Tune dropout; do not use it as a universal fix
Weight decay	Penalizes large parameter values	Overfit weights or poor generalization	Tune regularization strength
Early stopping	Stops when validation performance stops improving	Validation metric degrades for several checks	Restore best checkpoint and confirm validation design

Regularization helps reduce overfitting. Dropout randomly masks activations during training; weight decay discourages unnecessarily large weights; data augmentation creates altered but label-preserving examples; and early stopping selects a checkpoint based on validation performance. These techniques address different causes and may be combined. They cannot compensate for a fundamentally invalid split or missing coverage of important real-world cases.

Training curves are diagnostic evidence. If training loss decreases while validation loss increases, the network may be overfitting. If both losses remain high, the model may be underfitting, the learning rate may be unsuitable, the data may be noisy, or the target may be difficult to predict. If validation loss suddenly changes, inspect the data pipeline and evaluation procedure before immediately redesigning the architecture.

7. Evaluation metrics and threshold decisions

Evaluation metrics translate model behaviour into measurements, but no single metric captures every operational objective. Classification tasks often use a confusion matrix to separate true positives, true negatives, false positives, and false negatives. Accuracy measures the fraction of all predictions that are correct. Precision measures the proportion of predicted positives that are truly positive. Recall measures the proportion of actual positives detected. F1 is the harmonic mean of precision and recall, balancing the two when both matter.

Accuracy can be misleading when the positive class is rare. Suppose only 1 in 100 transactions is fraudulent. A model that predicts “not fraud” for every transaction achieves 99% accuracy but detects no fraud at all. Precision and recall reveal this failure immediately. ROC-AUC summarizes how well scores rank positive examples above negative examples across thresholds; precision-recall curves are often more informative when positives are rare. The operational threshold should be selected using the costs of missed cases, false alarms, review capacity, and calibration needs.

Metric	Interpretation	Useful when	Potential pitfall
Accuracy	Share of all predictions that are correct	Classes are reasonably balanced and errors have comparable costs	Can hide failure on a rare class
Precision	Of predicted positives, the share that are truly positive	False positives are costly	Can be high while many true positives are missed
Recall	Of actual positives, the share detected	False negatives are costly	Can be increased by predicting more positives
F1 score	Harmonic mean of precision and recall	A single balance measure is needed	Ignores true negatives and business costs directly
ROC-AUC	Ranking ability across false-positive rates and true-positive rates	Comparing score ranking across thresholds	May appear optimistic on highly imbalanced data
MAE	Average absolute numeric prediction error	Errors should be interpreted in target units	Treats errors linearly and underweights very large errors
RMSE	Square root of mean squared error	Large errors should be penalized more heavily	Sensitive to outliers
R-squared	Fraction of target variance explained relative to a mean baseline	Summarizing regression fit	Can be negative on test data; not a direct error-unit measure

Example threshold trade-off

A medical screening model assigns each patient a risk score. Lowering the positive threshold usually increases recall because more patients are flagged, but it can also reduce precision by producing more false alarms. Raising the threshold may reduce unnecessary follow-up, but more patients at risk may be missed. The right threshold is therefore a decision under constraints, not simply the default value used by a software library. The model should also be evaluated across relevant subgroups and across time to detect unequal errors or deterioration.

For regression, MAE and RMSE should be reported in context. An average error of 5 may be acceptable for a 10,000-unit prediction but unacceptable for a target measured in seconds. R-squared complements error metrics by comparing against a mean-prediction baseline, but it does not replace checks for biased residuals, subgroup errors, and out-of-range predictions.

8. Unsupervised learning and representation

Unsupervised learning searches for structure when a dataset has no provided target labels. Clustering methods assign examples to groups based on similarity or density; dimensionality reduction creates a lower-dimensional representation of the data; anomaly detection identifies observations that differ strongly from expected patterns. These methods are often used for exploration and as components of larger workflows, but their output still needs interpretation and validation against the purpose of the project.

K-means partitions numeric examples into a chosen number of clusters by repeatedly assigning each point to a nearby centroid and updating the centroids. It is relatively simple and efficient, but results depend on the choice of K and can be sensitive to feature scaling and outliers. It prefers roughly compact clusters and does not naturally represent every cluster shape. DBSCAN instead groups dense regions and can label isolated points as noise; it can identify irregular cluster shapes but depends on neighbourhood and minimum-density parameters, which can be difficult when the dataset contains regions with different densities.

Method	Core idea	Strength	Caution
K-means	Assign points to the nearest of K centroids	Fast and straightforward for compact numeric clusters	Choose K; scale features; outliers influence centroids
DBSCAN	Connect dense neighbourhoods and mark sparse points as noise	Can find irregular clusters and identify noise	Sensitive to distance scale and density settings
Hierarchical clustering	Build a tree of merges or splits	Dendrogram supports inspection at several granularities	Can be expensive for large datasets; linkage choice matters
PCA	Project onto directions explaining maximal variance	Useful linear compression and decorrelation	High variance is not always high predictive relevance
t-SNE / UMAP	Create low-dimensional views that preserve selected neighbourhood structure	Useful for visual exploration of complex data	Visual distances and apparent clusters can be misleading
Isolation Forest	Isolate unusual points through randomized partitioning	Practical anomaly-detection baseline	Anomaly scores require thresholding and domain review

Dimensionality reduction helps when many features are correlated or visualization is needed. Principal Component Analysis (PCA) creates linear combinations called principal components that capture decreasing amounts of total variance. The first component explains the largest possible share of variance subject to the projection constraint; each following component captures as much remaining variance as possible while being orthogonal to previous components. PCA is unsupervised, so a low-variance feature that is essential to predicting the target may be discarded if reduction is applied without considering downstream performance.

Cluster validation is not as simple as checking classification accuracy because there may be no ground-truth labels. The silhouette score compares how close a point is to its own cluster with its closeness to the nearest alternative cluster. A high score suggests separation under the chosen distance measure, but it does not prove that the groups are useful. Analysts should also inspect cluster sizes, stability, domain meaning, and whether the groups lead to actionable decisions.

9. Deployment, monitoring, and practical case studies

A trained model becomes useful only when it is integrated into a reliable workflow. A typical deployment pipeline validates incoming data, applies the same preprocessing used during training, obtains model predictions, records appropriate telemetry, and returns results within the latency and availability requirements. Models and preprocessing steps should be versioned together because a new encoder, scaler, or feature ordering can make an otherwise correct model produce invalid predictions.

Production data can differ from the data used to train a model. Data drift refers to changes in input distributions, while concept drift refers to changes in the relationship between inputs and the target. For example, shopping behaviour may change during a major sale, or fraud patterns may adapt to a detection system. Monitoring can track missing-value rates, category frequencies, feature ranges, latency, prediction distributions, and delayed performance metrics once ground-truth labels become available. A drift signal is an invitation to investigate; it is not automatically proof that the model must be retrained.

Case study A: Predicting customer churn

A subscription service wants to predict which accounts may cancel in the next 30 days. Useful features might include tenure, recent usage, support interactions, billing changes, and subscription type, provided each is available at scoring time. The target is defined using a fixed horizon. A temporal split is appropriate if the model will predict future churn from past observations. The team begins with logistic regression, compares it with a tree ensemble, assesses recall and precision at the available retention-team capacity, and checks whether performance is stable across customer segments.

Case study B: Detecting manufacturing defects

A factory collects images of products on a production line. A defect classifier is trained on labeled examples of acceptable and faulty products. Because images from the same batch or physical item can be highly similar, randomly placing near-duplicate images in both training and test sets can inflate evaluation. A group-aware split by batch, machine, or production day may provide a more realistic assessment. The team should also report false negatives because an undetected defect may reach a customer, while false positives increase manual inspection work.

Case study C: Forecasting energy demand

An energy provider forecasts hourly demand using past consumption, calendar information, weather forecasts, and operational variables. The evaluation must preserve time order and must use the weather forecast that would actually have been available at prediction time, not realized future weather. MAE helps communicate average absolute error in demand units; RMSE emphasizes unusually large misses. The team should evaluate peak-load periods separately because average performance may conceal the errors that matter most for grid planning.

Operational concern	What to record or test	Why it matters
Reproducibility	Dataset version, split, random seed, code and model version	Allows consistent comparisons and rollback
Data quality	Schema, missingness, ranges, new categories	Catches upstream changes before prediction
Service health	Latency, error rate, throughput, resource use	Protects user experience and availability
Model performance	Delayed labels, task metrics, subgroup metrics	Detects deterioration and uneven outcomes
Governance	Purpose, limitations, access, retention policy	Supports responsible and auditable operation

Final takeaway: model development is an iterative process of framing a measurable task, preparing data without leakage, training baselines, evaluating with metrics that match the cost of errors, inspecting failures, and monitoring behaviour after deployment. Model complexity is useful only when it produces a robust improvement under an appropriate evaluation design.

Glossary

Feature: an input variable. Target: the value to predict. Hyperparameter: a training or model setting selected by the developer. Overfitting: learning patterns that do not generalize. Leakage: unintended access to information that should not be available during training or prediction. Drift: change in input data or target relationships over time.